

第五章 优化程序性能

目录

1、优化方法

1.1 编译器层面：GCC 优化指令

1.2 普遍有用的优化方法

1.3 编译器优化的局限

- 阻碍编译器优化的因素

2、表示程序性能

3、提高并行性

3.1 理解现代处理器

3.2 理解内存性能

3.2 循环展开

1.1 GCC 优化指令

- 1) **Og**:默认配置，不优化。
- 2) **O1**:编译器试图优化代码大小和执行时间，它主要对代码的分支，常量以及表达式等进行优化，但不执行任何会占用大量编译时间的优化。
- 3) **O2**:GCC 执行几乎所有不包含时间和空间权衡的优化（比如，尝试更多的寄存器级的优化以及指令级的优化）。与**-O**相比，此选项增加了编译时间，但提高了代码的效率
- 4) **O3**:比**-O2**更优化，对于**-O3**编译选项，在**-O2**的基础上，打开了更多的优化项（比如，使用伪寄存器网络，普通函数的内联 inline，以及针对循环的更多优化）。不过可能导致编译出来的二级制程序不能 debug

1.2 普遍有用的优化方法

1) 减少循环低效调用

把 `combine1` 中循环的结束条件中 `vec_length` 函数放到函数之外

```
1 void combine2(vec_ptr v, data_t *dest) {
2
3     *dest = IDENT;
4
5     long length = vec_length(v);
6
7     for (long i = 0; i < length; i++) {
8         data_t val;
9         get_vec_element(v, i, &val);
10        *dest = *dest OP val;
11    }
12
13 }
```

2) 减少程序调用

可以直接操作数组来获取元素而不是通过调用函数

```
1 data_t *get_vec_start(vec_ptr v) {
2     return v->data;
3 }
4
5 void combine3(vec_ptr v, data_t *dest) {
6
7     *dest = IDENT;
8
9     long length = vec_length(v);
10    data_t *data = get_vec_start(v);
11
12    for (long i = 0; i < length; i++) {
13        *dest = *dest OP data[i];
14    }
15
16 }
```

3) 减少不必要的内存引用

`combine3` 循环中每次都要调用 `dest` 指针所指向的内存

可以通过引入中间变量而消除这种不必要的内存引用

```
1 void combine4(vec_ptr v, data_t *dest) {  
2  
3     long length = vec_length(v);  
4     data_t *data = get_vec_start(v);  
5  
6     data_t res = IDENT;  
7  
8     for (long i = 0; i < length; i++) {  
9         res = res OP data[i];  
10    }  
11  
12    *dest = res;  
13 }
```

4) 循环展开 (2x1)

5) 使用分离的累加器 (2x2)

6) 带重组的循环展开 (2x1a)

- (4) (5) (6) 见 3.3 部分

1.3 编译器优化的局限

1) 单个文件过程间分析

2) 编译器优化必须保守

a. 函数调用

妨碍优化的因素: 函数调用

■ 为什么编译器不能将 `strlen` 从内层循环中移出呢?

- 函数可能有副作用
 - 例如: 每次被调用都改变全局变量/状态
- 对于给定的参数, 函数可能返回不同的值
 - 依赖于全局状态/变量的其他部分
 - 函数 `lower` 可能与 `strlen` 相互作用

■ Warning:

- 编译器将函数调用视为黑盒
- 在函数附近进行弱优化

■ 补救措施:

- 使用 `inline` 内联函数
 - 用 `-O1` 时GCC这样做, 但局限于单一文件之内
- 程序员自己做代码移动

```
size_t lencnt = 0;
size_t strlen(const char *s)
{
    size_t length = 0;
    while (*s != '\0') {
        s++; length++;
    }
    lencnt += length;
    return length;
}
```

第一点: 该函数每次调用可能会改变全局变量, 编译器无法将处于循环中的函数调用到函数外

第二点: 就算每次输入参数相同, 可能返回值调用全局变量或与其他函数相互作用而不同

b. 内存别名使用

内存别名使用Memory Aliasing

```
/* Sum rows is of n X n matrix a
and store in vector b */
void sum_rows1(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        b[i] = 0;
        for (j = 0; j < n; j++)
            b[i] += a[i*n + j];
    }
}
```

```
double A[9] =
{ 0, 1, 2,
  4, 8, 16,
  32, 64, 128};
double B[3];
sum_rows1(A, A+3, 3);
```

- 代码每次循环都更新 **b[i]**
- 必须考虑这些更新会影响程序行为的可能性

Value of **b**:

init: [4, 8, 16]

i = 0: [3, 8, 16]

i = 1: [3, 22?, 16?]

i = 2: [3, 22?, 224?]

原本期望值 3, 28, 224

传入的两个指针 **a** (**A**)，**b** (**A+3**) 在遍历过程会访问同一段内存区域，编译器无法判断是否是编写本意，因此无法用局部变量代替全局变量进行累加操作

移除内存别名

```
/* Sum rows is of n X n matrix a
and store in vector b */
void sum_rows2(double *a, double *b, long n) {
    long i, j;
    for (i = 0; i < n; i++) {
        double val = 0;
        for (j = 0; j < n; j++)
            val += a[i*n + j];
        b[i] = val;
    }
}
```

- 而且不需要存储中间结果

```
# sum_rows2 inner loop
.L10:
    addsd (%rdi), %xmm0 # FP load + add
    addq $8, %rdi
    cmpq %rax, %rdi
    jne .L10
```

2、CPE：每个元素的周期数（cycle per element）：CPE 越小越好

表示程序性能

- CPE：每个元素的周期数(Cycles Per Element)
- 表示向量或列表操作的程序性能的方便方式
- $\text{Length} = n$
- In our case: $\text{CPE} = \text{cycles per OP}$
- $T = \text{CPE} * n + \text{经常开销(Overhead)}$
 - CPE 是线的斜率slope

3.1 理解现代处理器

超标量处理器（大多数现代 **cpu**）：一个周期执行多条指令

好处：处理器直接利用程序指令级并行性

3.3 循环展开

下面代码被称作 `2 * 1` 的循环展开

```
1 void combine5(vec_ptr v, data_t *dest) {
2
3     long length = vec_length(v);
4     data_t *data = get_vec_start(v);
5
6     data_t res = IDENT;
7
8     long i;
9     for (i = 0; i < length; i += 2) {
10         res = (res OP data[i]) OP data[i + 1];
11     }
12
13     for (; i < length; i++) res = (res OP data[i]);
14
15     *dest = res;
16 }
```

结果：1) 对于整数加法 CPE 降低：降低循环运算开销

2) 对于其他运算情况 CPE 没有降低：达到延迟界限，简单循环展开无法继续降低 CPE

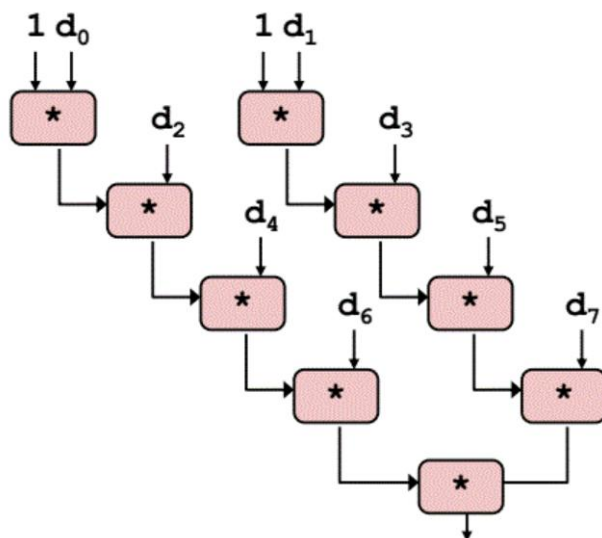
仍是顺序依赖，没有改变关键路径长度

使用 2 个临时变量来存储，从而提高并行性，这被称作 2×2 的循环展开

```
1 void combine6(vec_ptr v, data_t *dest) {
2
3     long length = vec_length(v);
4     data_t *data = get_vec_start(v);
5
6     data_t acc0 = IDENT;
7     data_t acc1 = IDENT;
8
9
10    long i;
11    for (i = 0; i < length; i += 2) {
12        acc0 = acc0 OP data[i];
13        acc1 = acc1 OP data[i + 1];
14    }
15
16    for (; i < length; i++) acc0 = (acc0 OP data[i]);
17
18    *dest = acc0 OP acc1;
19 }
```

分离的累加器

```
x0 = x0 OP d[i];
x1 = x1 OP d[i+1];
```



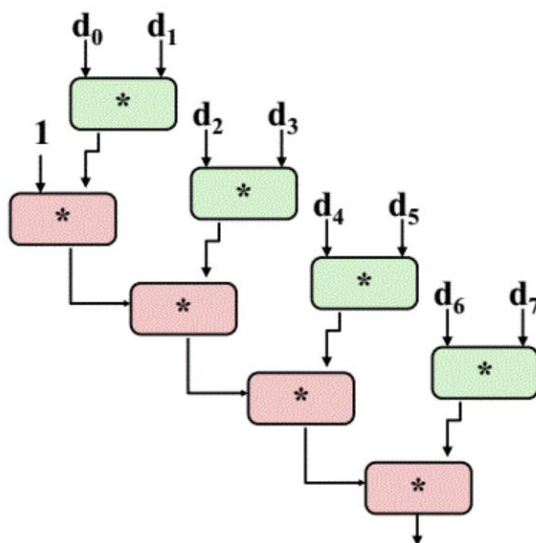
- 什么改变了:
 - 两个独立的操作的“流水”
- 整体性能
 - N 个元素, 每个操作 D 个周期延迟
 - 应为 $(N/2+1)*D$ cycles:
 $CPE = D/2$
 - CPE 与预测匹配!

根据结合律，尽量减少关键路径的长度，这称之为 $2 * 1a$ 变换

```
1 void combine7(vec_ptr v, data_t *dest) {
2
3     long length = vec_length(v);
4     data_t *data = get_vec_start(v);
5
6     data_t res = IDENT;
7
8     long i;
9     for (i = 0; i < length; i += 2) {
10         res = res OP (data[i] OP data[i + 1]);
11     }
12
13     for (; i < length; i++) res = (res OP data[i]);
14
15     *dest = res;
16 }
```

重组的计算

$x = x \text{ OP } (d[i] \text{ OP } d[i+1]);$



■ 什么改变了:

- 下一个循环的操作可以早一些开始 (没有依赖)

■ 整体性能

- N 个元素, 每个操作 D 个周期延迟
- $(N/2+1)*D$ cycles:
 $CPE = D/2$

方法	Integer		Double FP	
操作 OP	+	*	+	*
Combine1 未优化	22.68	20.02	19.98	20.18
Combine1 -O1	10.12	10.12	10.17	11.14
Combine4	1.27	3.01	3.01	5.01
Unroll 2x1	1.01	3.01	3.01	5.01
Unroll 2x1a	1.01	1.51	1.51	2.51
Unroll 2x2	0.81	1.51	1.51	2.51
延迟界限	1.00	3.00	3.00	5.00
吞吐量界限	0.50	1.00	1.00	0.50

图-7 循环展开各版本函数 CPE 值比较

Haswell 架构 cpu

延迟：执行该运算所需时钟周期数

发射：两个相同运算之间间隔的时钟周期数

容量：每次能同时发射多少个这样的操作

延迟界限（单位：CPE）严格按照顺序完成合并运算的函数所需最小 CPE

吞吐量界限：当所有参与运算的功能单元都完全利用起来后达到的最小 CPE 值

吞吐量界限 = 发射/容量